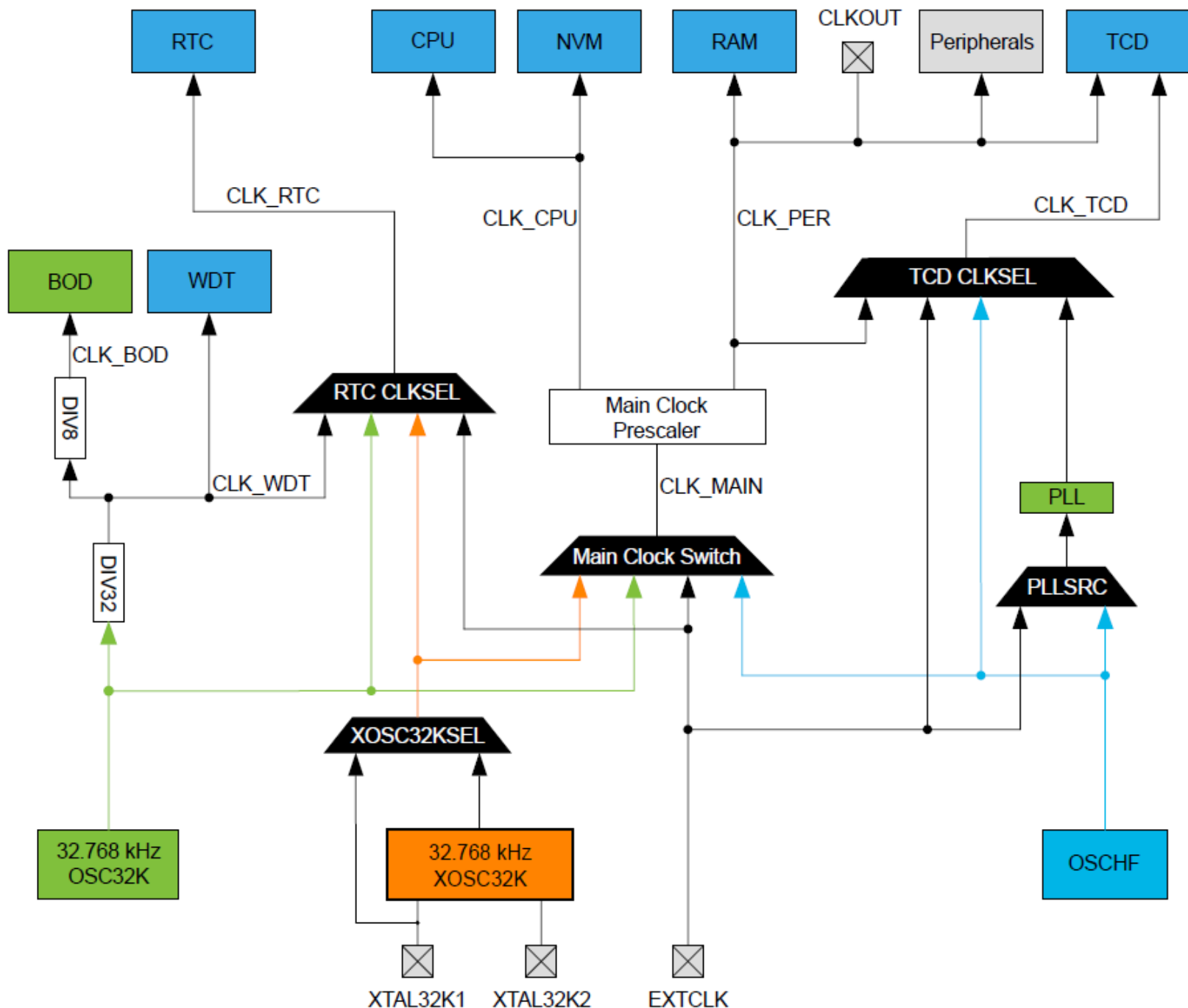


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 3 - Clock System (CLKCTRL)



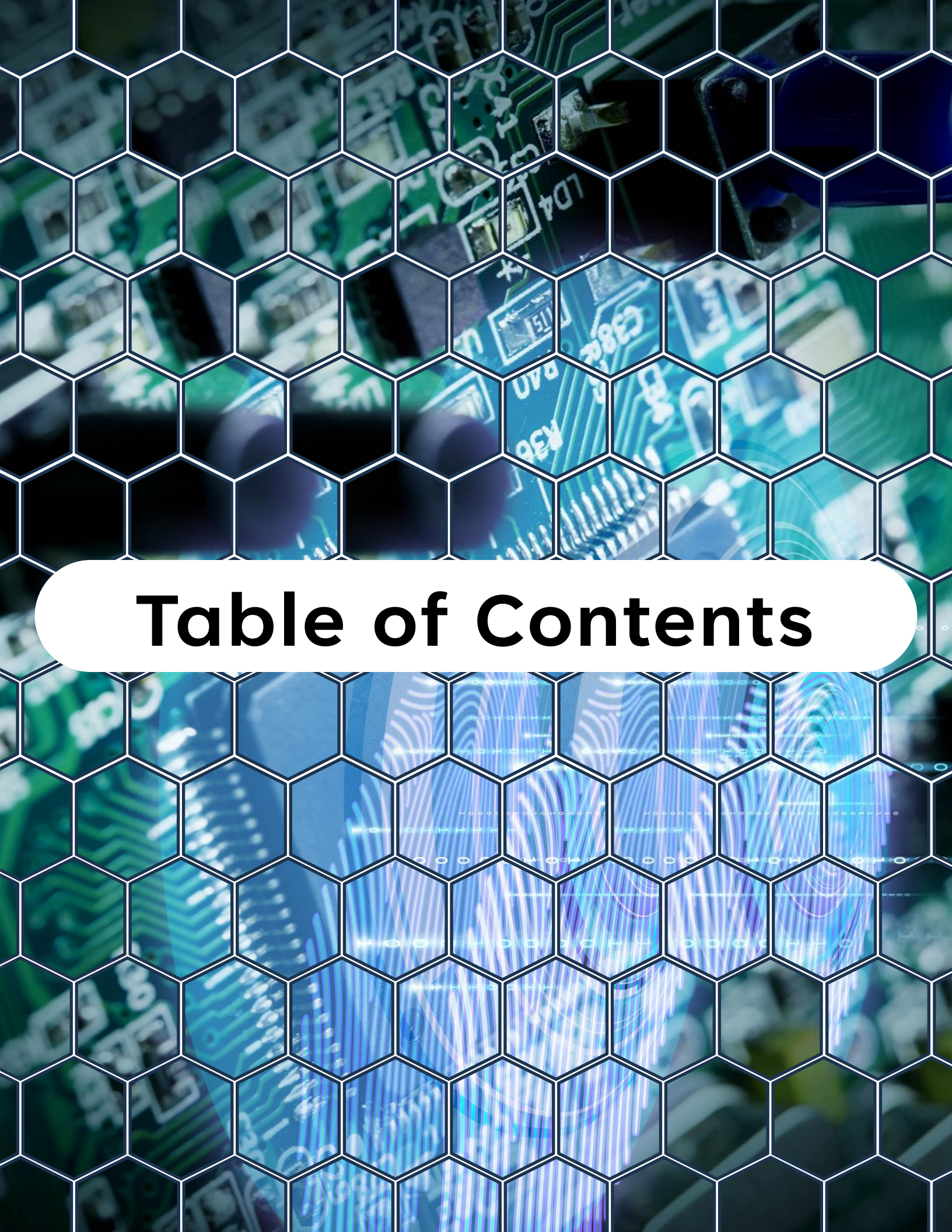
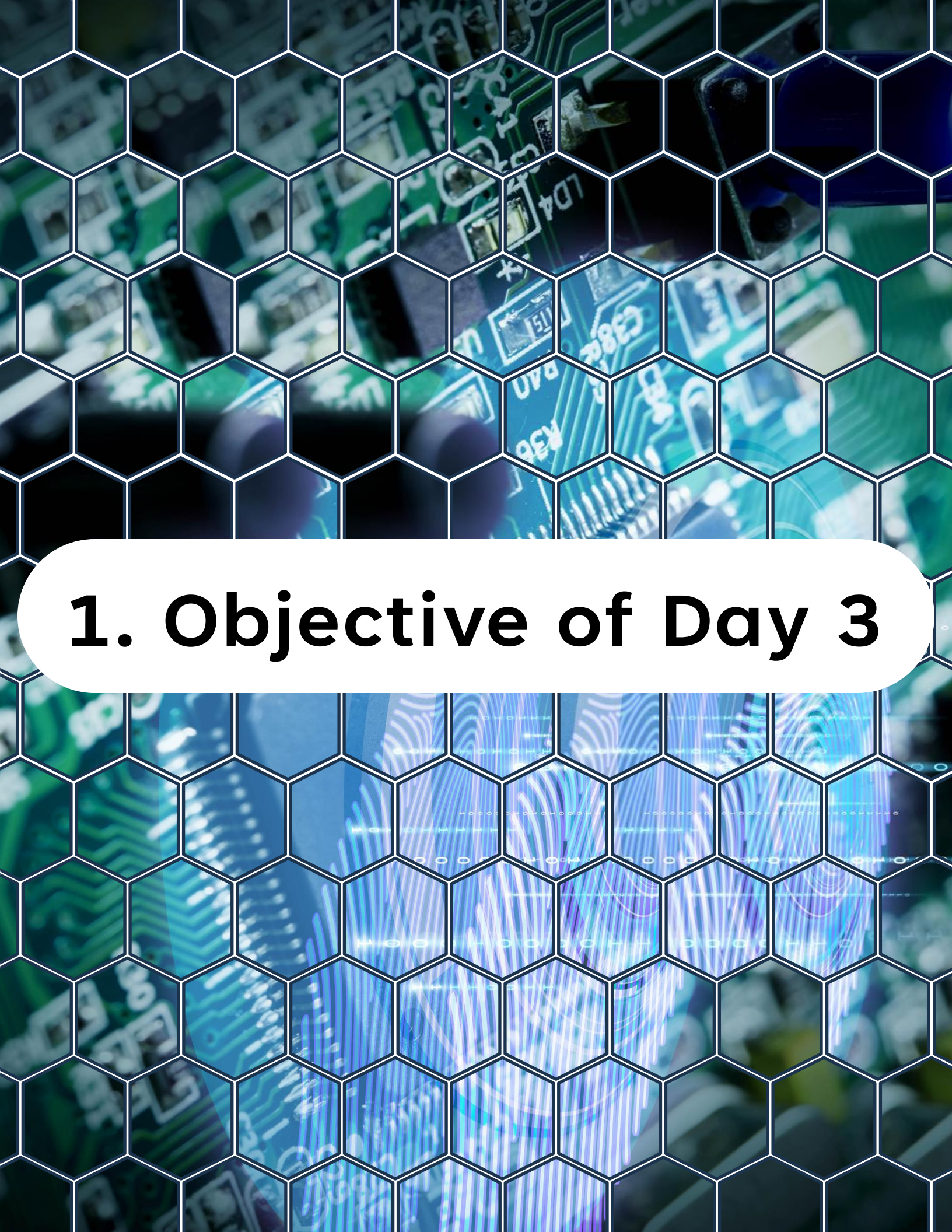


Table of Contents

Table of Contents

1. Objective of Day 3
2. Clock System Overview
3. Clock Sources in AVR128DA48
4. Default Clock Behavior After Reset
5. CLKCTRL Peripheral Fundamentals
6. Main Clock Selection
7. Prescalers and Why They Matter
8. Peripheral Clock Domains
9. Power vs Performance Tradeoffs
10. Practical Lab - Proving the Clock Frequency
11. Debugging Clock Issues
12. What You Learned Today
13. What Comes Next



1. Objective of Day 3

1. Objective of Day 3

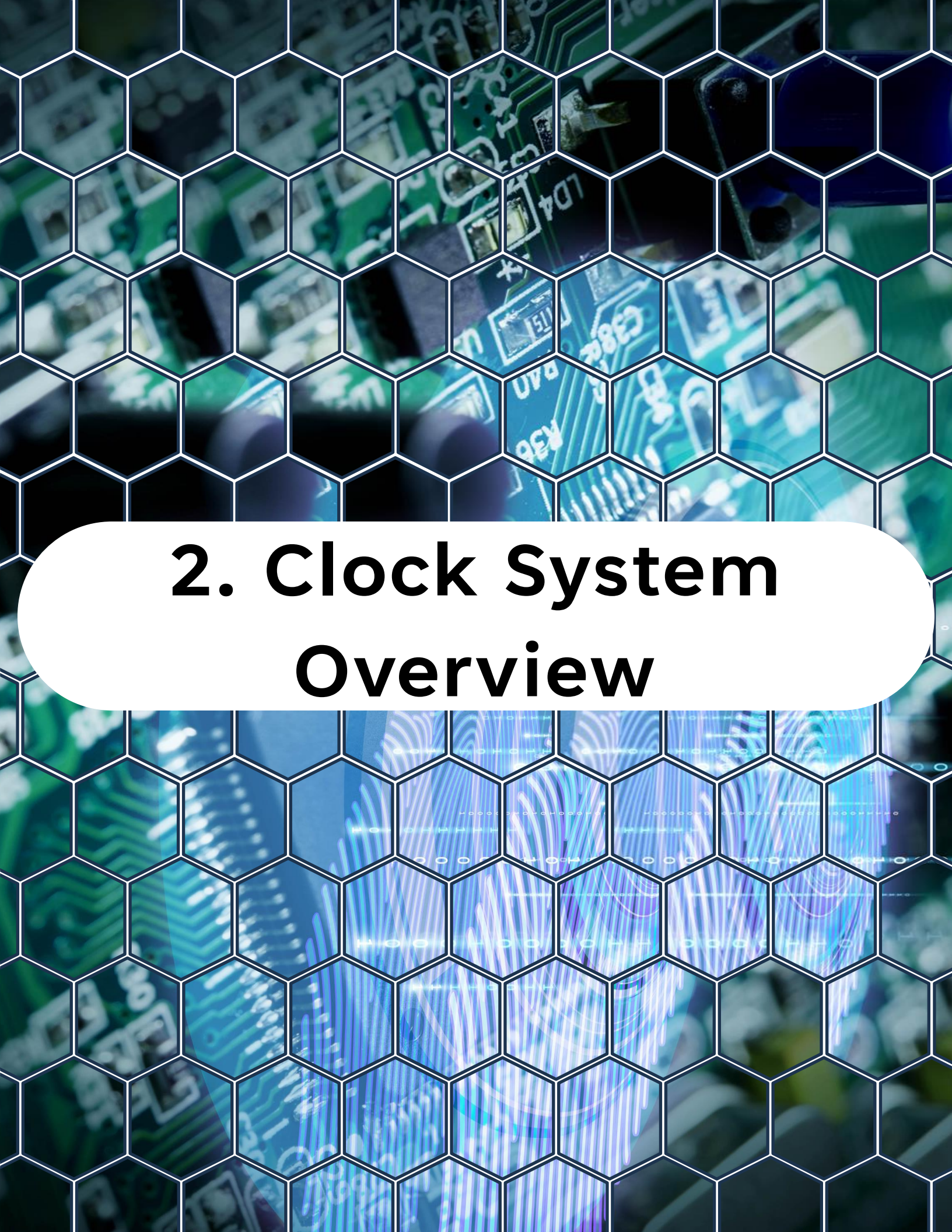
Today you will learn how time is created inside the MCU.

Every delay, baud rate, PWM frequency, ADC sampling rate, and timeout depends on the clock system. If you do not understand the clock tree, you cannot reason about timing, power consumption, or correctness.

By the end of this day, you will be able to:

- Understand where the CPU clock comes from
- Configure the main clock using CLKCTRL
- Predict execution speed and peripheral timing
- Verify clock configuration using real measurements

We are now officially leaving the "static" world and entering the "time-aware" world.



2. Clock System Overview

2. Clock System Overview

The **AVR128DA48** includes a flexible and modern clock system managed by the CLKCTRL peripheral.

At a high level, the clock system consists of:

Clock sources (oscillators)

Clock selection logic

Prescalers

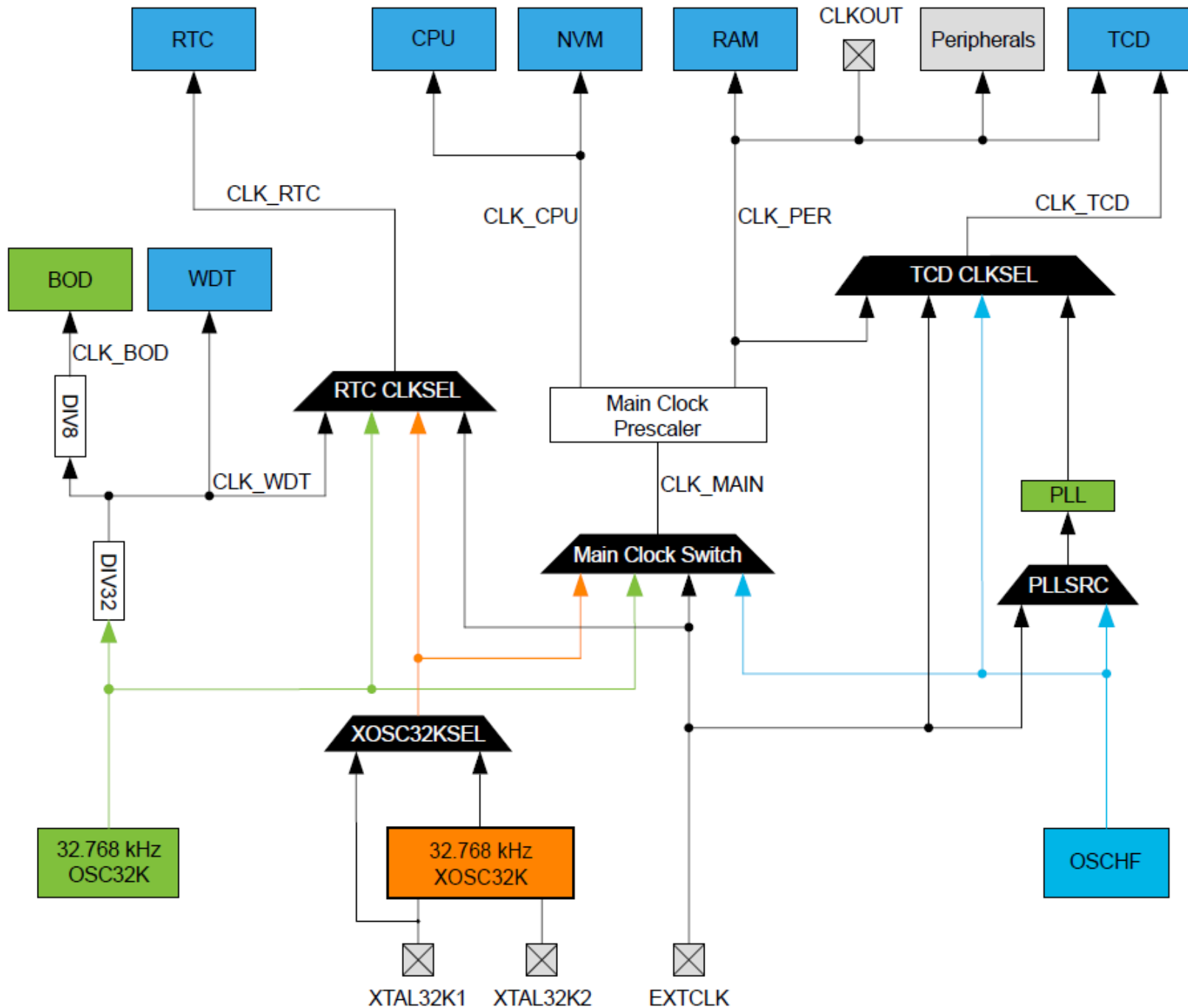
Peripheral clock distribution

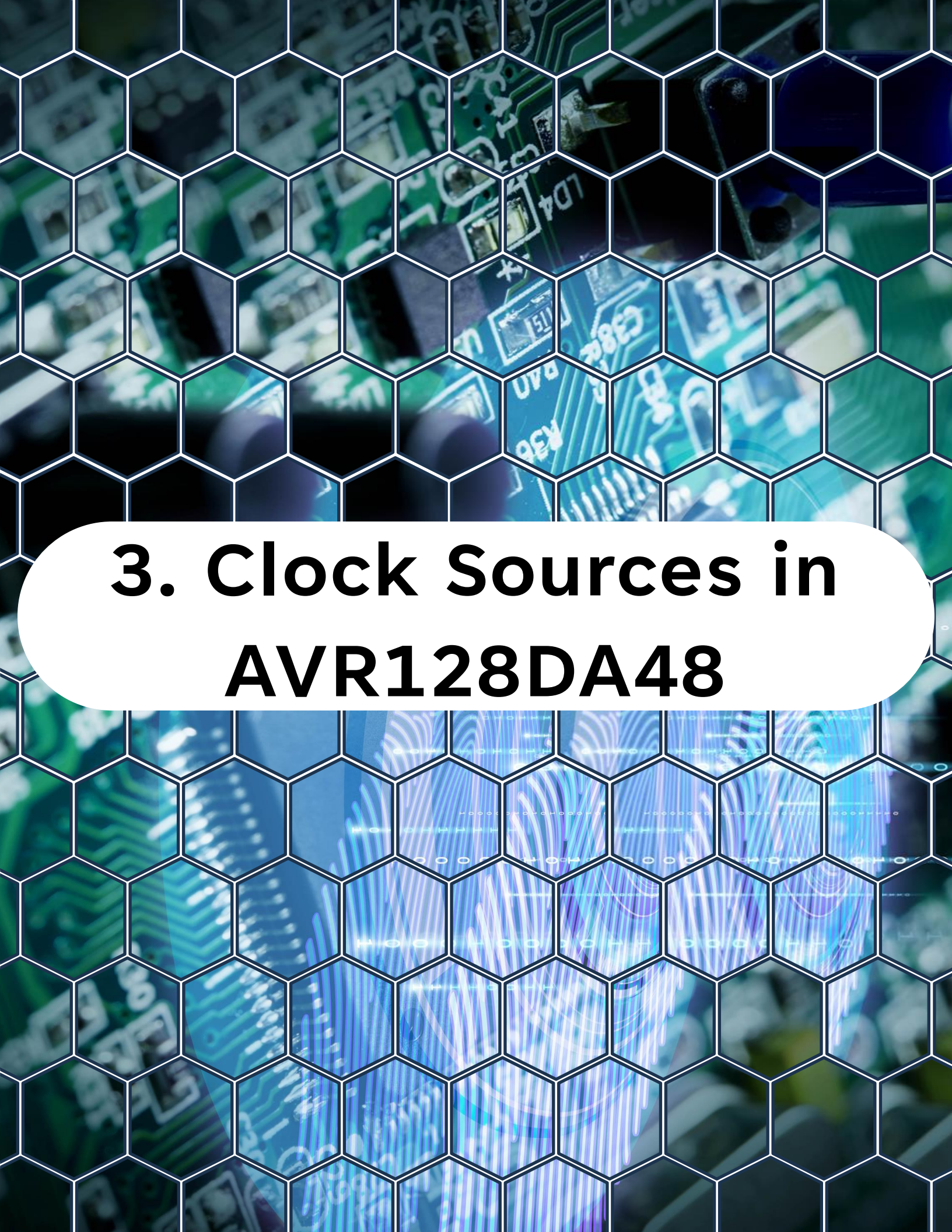
The CPU does not magically run at a frequency.

It is driven by a selected oscillator, optionally divided by prescalers, then distributed to peripherals.

2. Clock System Overview

CLKCTRL Block Diagram





3. Clock Sources in AVR128DA48

3. Clock Sources in **AVR128DA48**

The **AVR128DA48** provides multiple clock sources, the most important being:

Internal High-Frequency Oscillator (OSCHF)

- Factory calibrated
- Default clock source after reset
- Typical frequencies: 24 MHz, 20 MHz, 16 MHz, etc.
- Good accuracy for most applications

Internal Low-Frequency Oscillator (OSCULP32K)

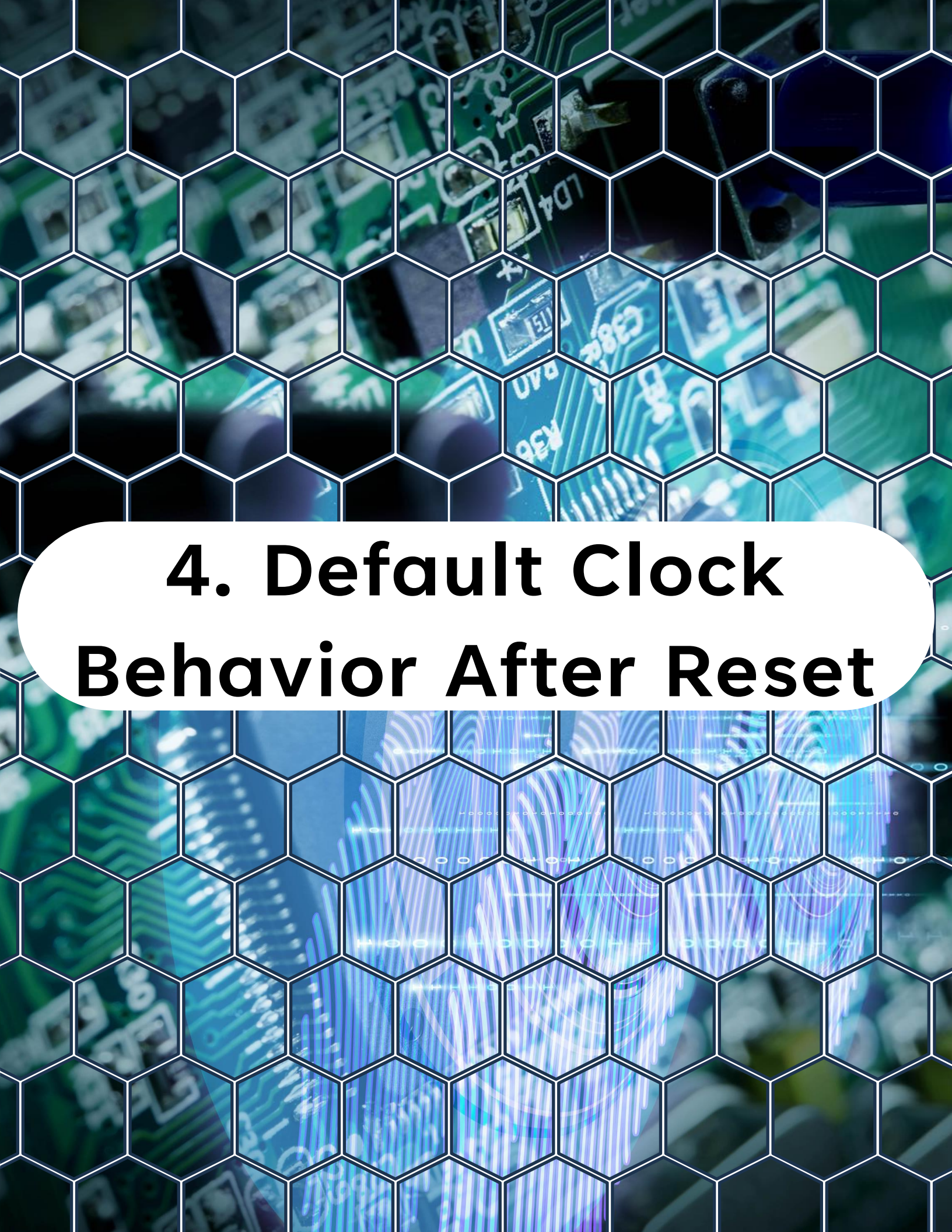
- 32.768 kHz ultra-low-power oscillator
- Used mainly by **RTC**
- Optimized for low power, not speed

3. Clock Sources in AVR128DA48

External Clock (optional)

- External crystal or clock input
- Used when higher accuracy is required
- Not mandatory for this course

For this course, we will rely primarily on the internal oscillators.



4. Default Clock Behavior After Reset

4. Default Clock Behavior After Reset

One critical concept: the MCU already runs before you write any code.

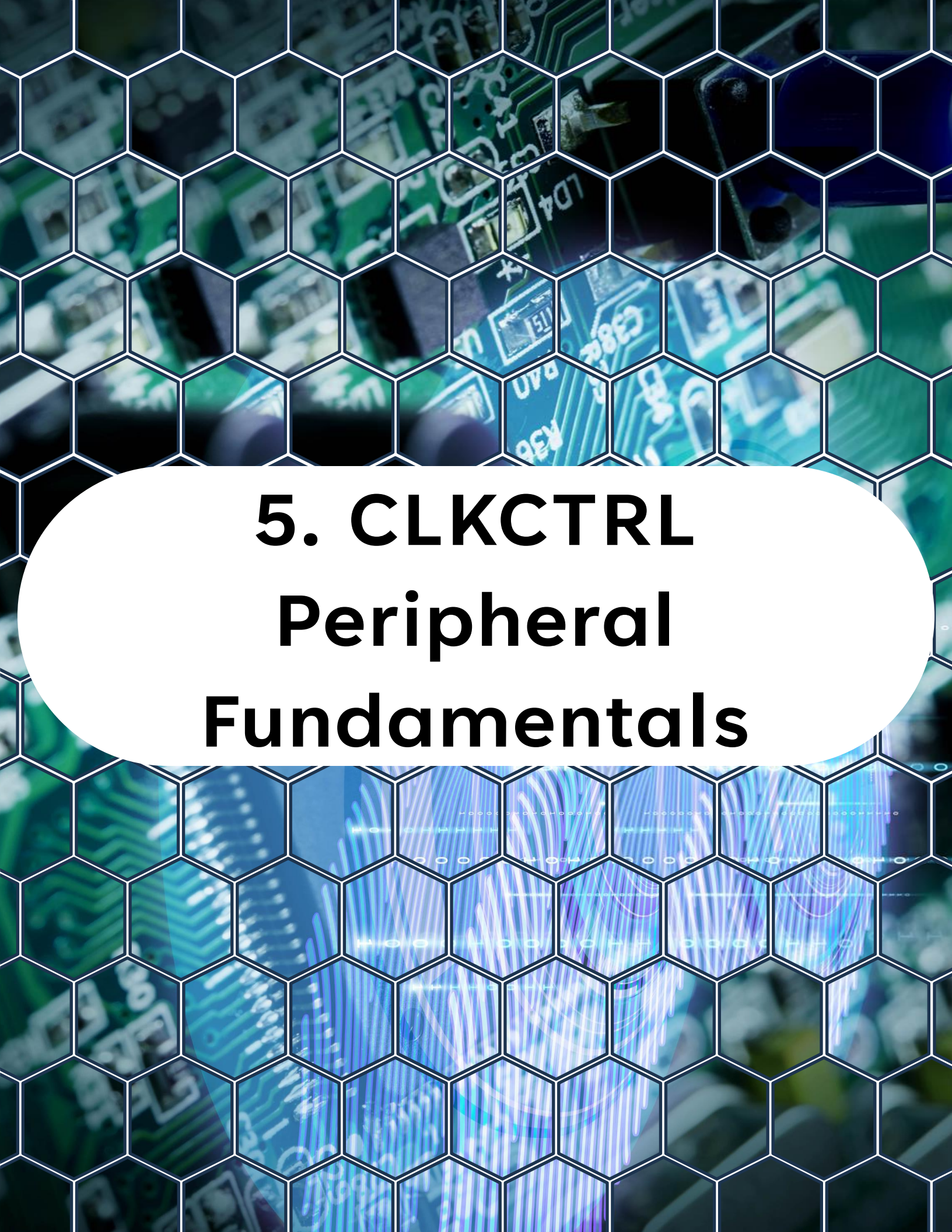
After reset:

- The internal high-frequency oscillator is enabled
- A default prescaler may be active
- The CPU starts executing immediately

This means:

- Your code is already timing-sensitive from the first instruction
- Clock configuration should be done early in `main()`

Never assume the clock frequency. Always configure it explicitly.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb-like texture. The text is centered within a white, rounded rectangular area in the middle of the slide.

5. CLKCTRL Peripheral Fundamentals

5. CLKCTRL Peripheral Fundamentals

The CLKCTRL peripheral controls:

- Which oscillator is used
- CPU clock prescaling
- Clock output configuration
- Peripheral clock enablement

Important registers you will encounter:

- **CLKCTRL.MCLKCTRLA**
- **CLKCTRL.MCLKCTRLB**
- **CLKCTRL.MCLKSTATUS**

You do not need to memorize them. You need to know how to read them.

5. CLKCTRL Peripheral Fundamentals

CLKCTRL Register Summary

Offset	Name	Bit Pos.								
0x00	MCLKCTRLA	7:0	CLKOUT				CLKSEL[3:0]			
0x01	MCLKCTRLB	7:0				PDIV[3:0]				PEN
0x02	MCLKLOCK	7:0								LOCKEN
0x03	MCLKSTATUS	7:0			PLLS	EXTS	XOSC32KS	OSC32KS	OSCHFS	SOSC
0x04 ... 0x07	Reserved									
0x08	OSCHFCTRLA	7:0	RUNSTDBY		FRQSEL[3:0]					AUTOTUNE
0x09	OSCHFTUNE	7:0	TUNE[7:0]							
0x0A ... 0x0F	Reserved									
0x10	PLLCTRLA	7:0	RUNSTDBY	SOURCE					MULFAC[1:0]	
0x11 ... 0x17	Reserved									
0x18	OSC32KCTRLA	7:0	RUNSTDBY							
0x19 ... 0x1B	Reserved									
0x1C	XOSC32KCTRLA	7:0	RUNSTDBY		CSUT[1:0]			SEL	LPMODE	ENABLE



6. Main Clock Selection

6. Main Clock Selection

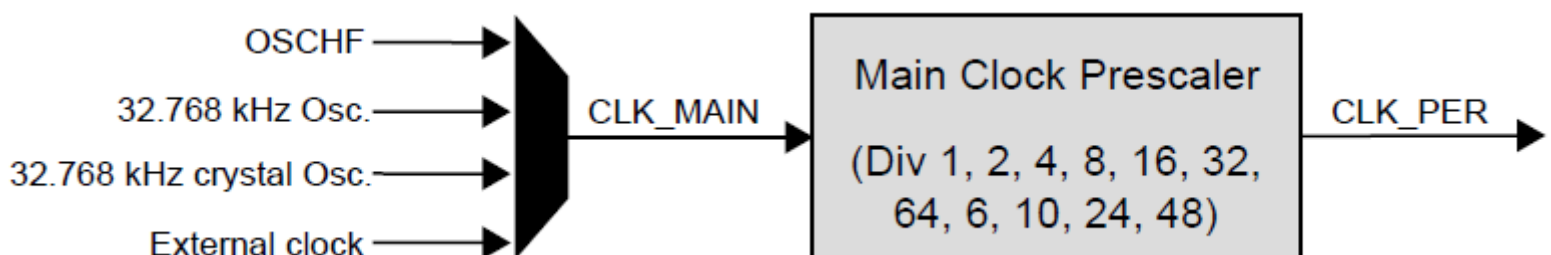
The CPU main clock (CLK_MAIN) is selected using **CLKCTRL** register.

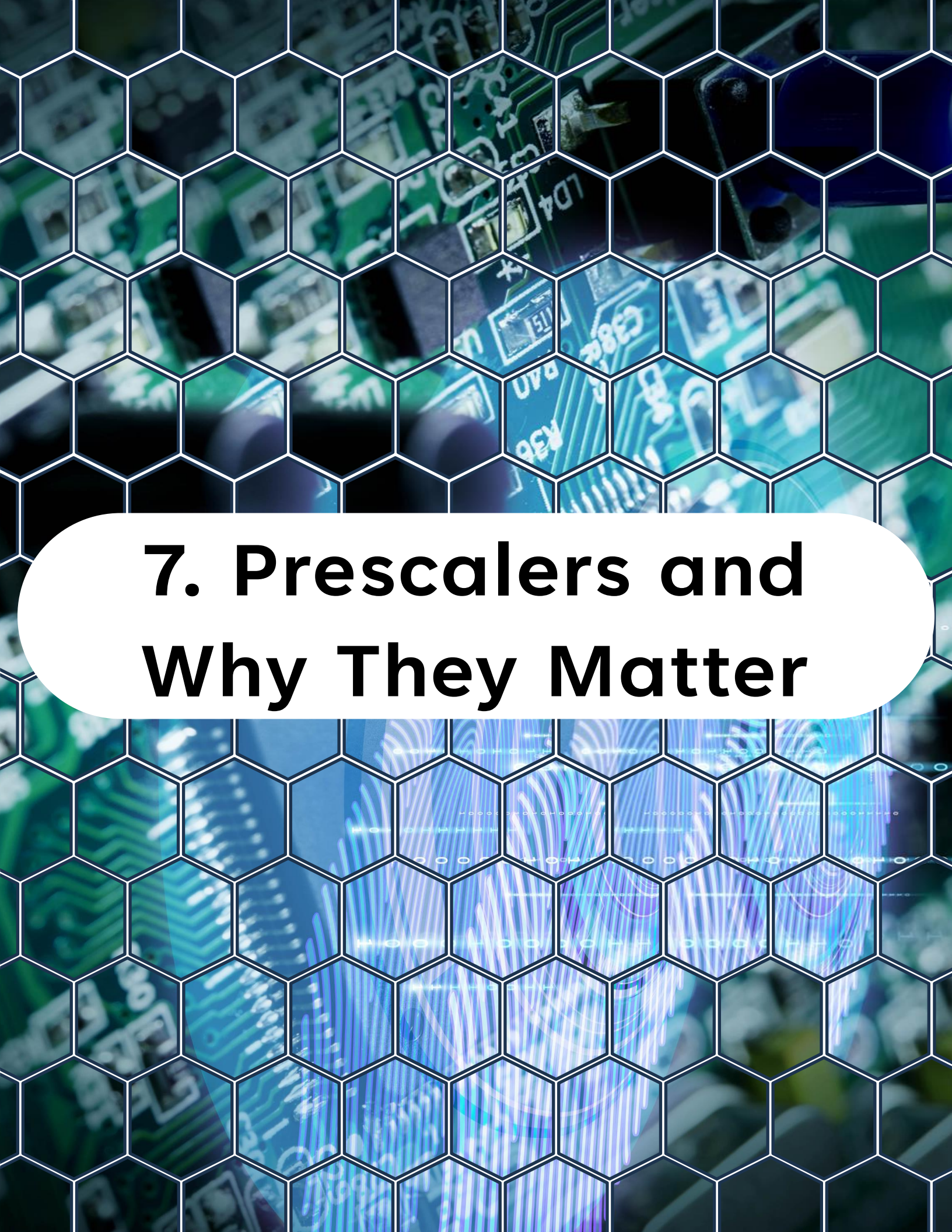
Key ideas:

- Only one main clock source drives the CPU at a time
- Switching clock sources must follow defined sequences
- Status flags indicate when a clock is stable

Clock switching is deterministic and safe if done correctly.

Main Clock and Prescaler





7. Prescalers and Why They Matter

7. Prescalers and Why They Matter

Prescalers divide the main clock frequency before it reaches the CPU.

Example:

- Oscillator = 20 MHz
- Prescaler = 2
- CPU frequency = 10 MHz

Why use prescalers?

- Reduce power consumption
- Match timing requirements
- Simplify peripheral calculations

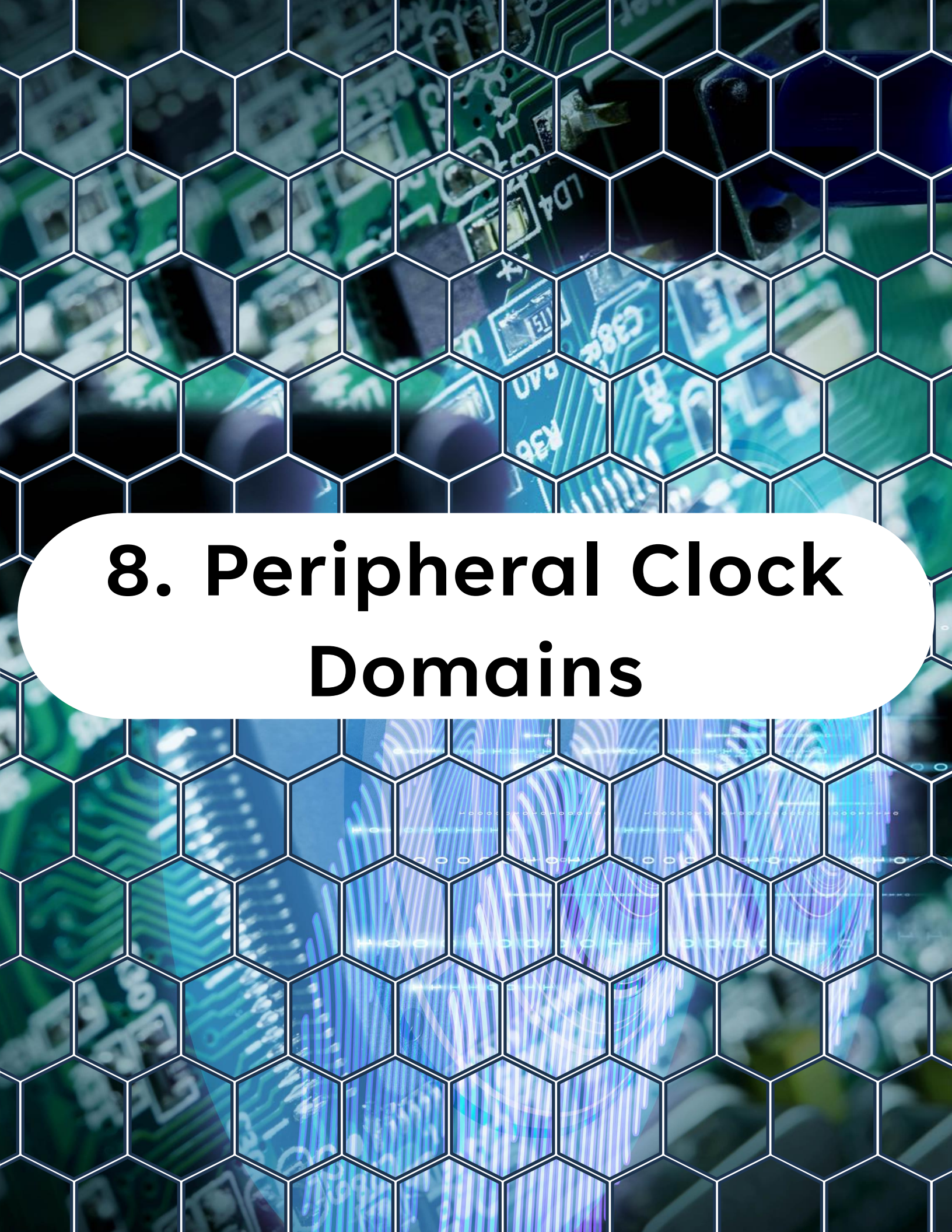
Prescalers affect:

- Instruction execution speed
- Timer tick rate

7. Prescalers and Why They Matter

- Communication baud rates

If timing looks wrong, prescalers are often the cause.



8. Peripheral Clock Domains

8. Peripheral Clock Domains

Not all peripherals run at the same speed.

The AVR128DA48 has multiple clock domains:

- CPU clock
- Peripheral clock
- RTC clock domain (often 32 kHz)

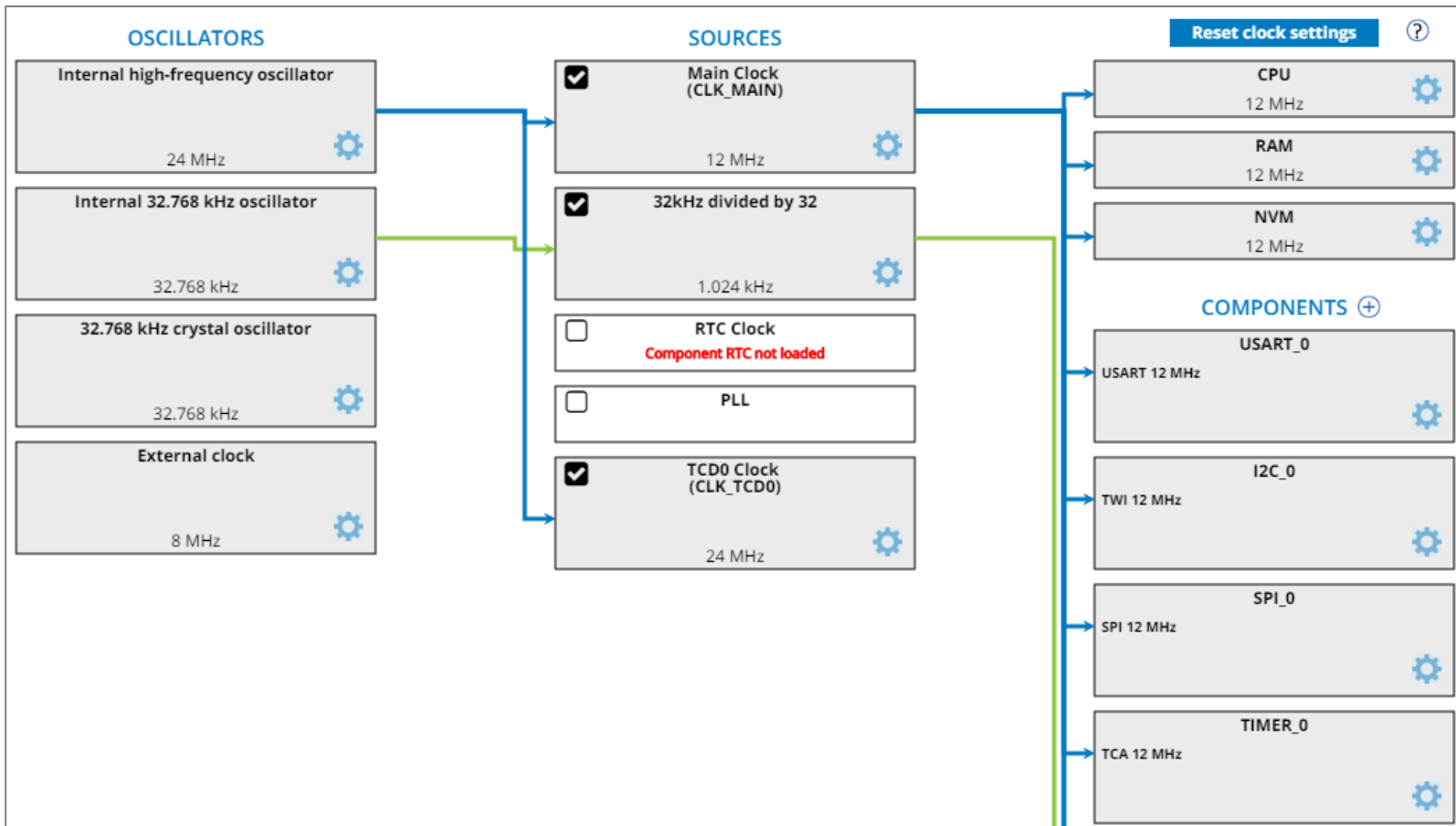
This separation allows:

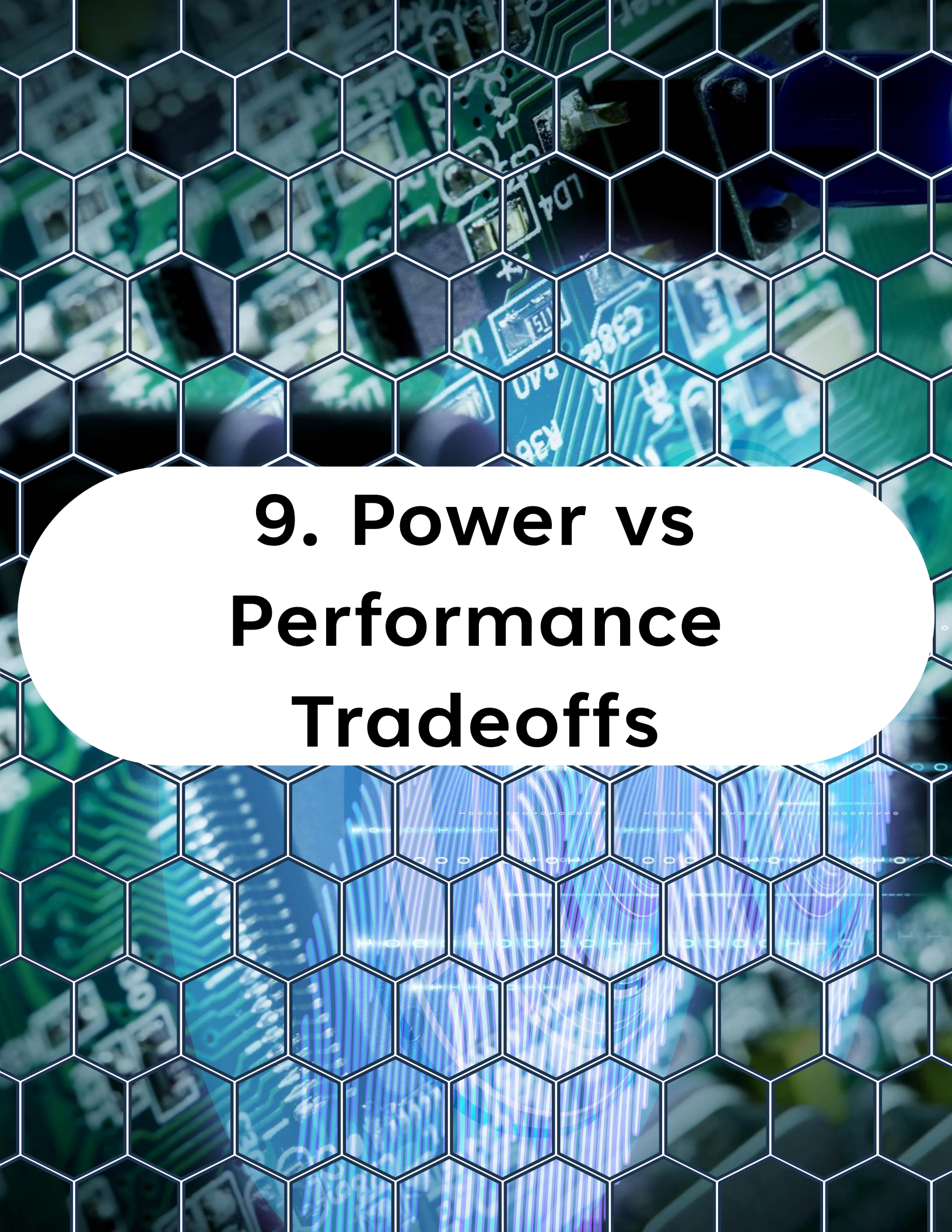
- Low-power peripherals to keep running while CPU sleeps
- Accurate timekeeping independent of CPU speed

Understanding this separation is essential when using RTC, ADC, or low-power modes.

8. Peripheral Clock Domains

Clock Domains



The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', 'U7A', and 'K3B'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb-like texture. In the center, a white rounded rectangle contains the title text.

9. Power vs Performance Tradeoffs

9. Power vs Performance Tradeoffs

Clock speed directly affects:

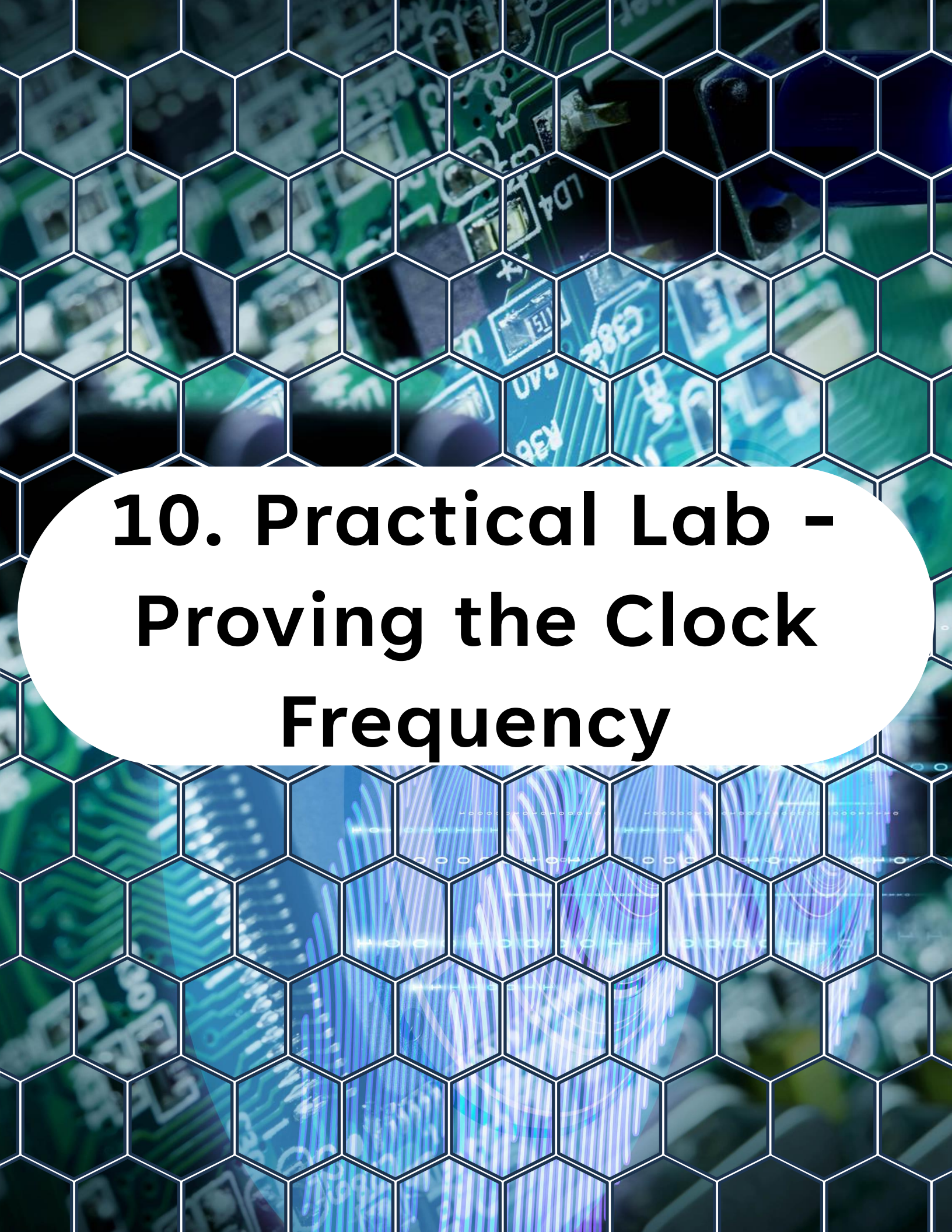
- Power consumption
- Heat generation
- EMI
- Timing resolution

Running faster is not always better.

Professional embedded systems often:

- Run fast only when needed
- Slow down or sleep when idle
- Use peripherals to offload CPU work

You will apply these principles later when combining timers, RTC, and sleep modes.



10. Practical Lab - Proving the Clock Frequency

10. Practical Lab - Proving the Clock Frequency

Lab Goal

Verify the CPU clock using a GPIO toggle and measurement.

This is a critical embedded skill: trust measurements, not assumptions.

Concept

Toggle a GPIO pin inside a loop

Measure the frequency with:

- Oscilloscope
- Logic analyzer
- Frequency counter

From the toggle rate, infer CPU speed.

10. Practical Lab - Proving the Clock Frequency

Step 1: Configure a GPIO Pin

Choose a known output pin connected to an LED or header.

Set it as output using the PORT registers.

```
/* Configure PC4 as output (push-pull). */
static void gpio_init_pc4_output(void)
{
    /* PC4 output */
    PORTC.DIRSET = PIN4_bm;

    /* Optional: start low */
    PORTC.OUTCLR = PIN4_bm;
}
```

10. Practical Lab - Proving the Clock Frequency

Step 2: Toggle the Pin in a Loop

Inside the main loop:

- Toggle the pin state
- Do nothing else

No delays. No timers. Pure execution speed.

```
// Application entry point
int main(void) {
    // Configure main clock
    clock_init_24mhz_presc2();

    // Configure PC4 as output
    gpio_init_pc4_output();

    /* Toggle as fast as possible.
     * Use OUTTGL to flip only the selected bit
     * without read-modify-write.
     */
    for (;;) {
        PORTC.OUTTGL = PIN4_bm;
    }
}
```

10. Practical Lab - Proving the Clock Frequency

Step 3: Measure the Output

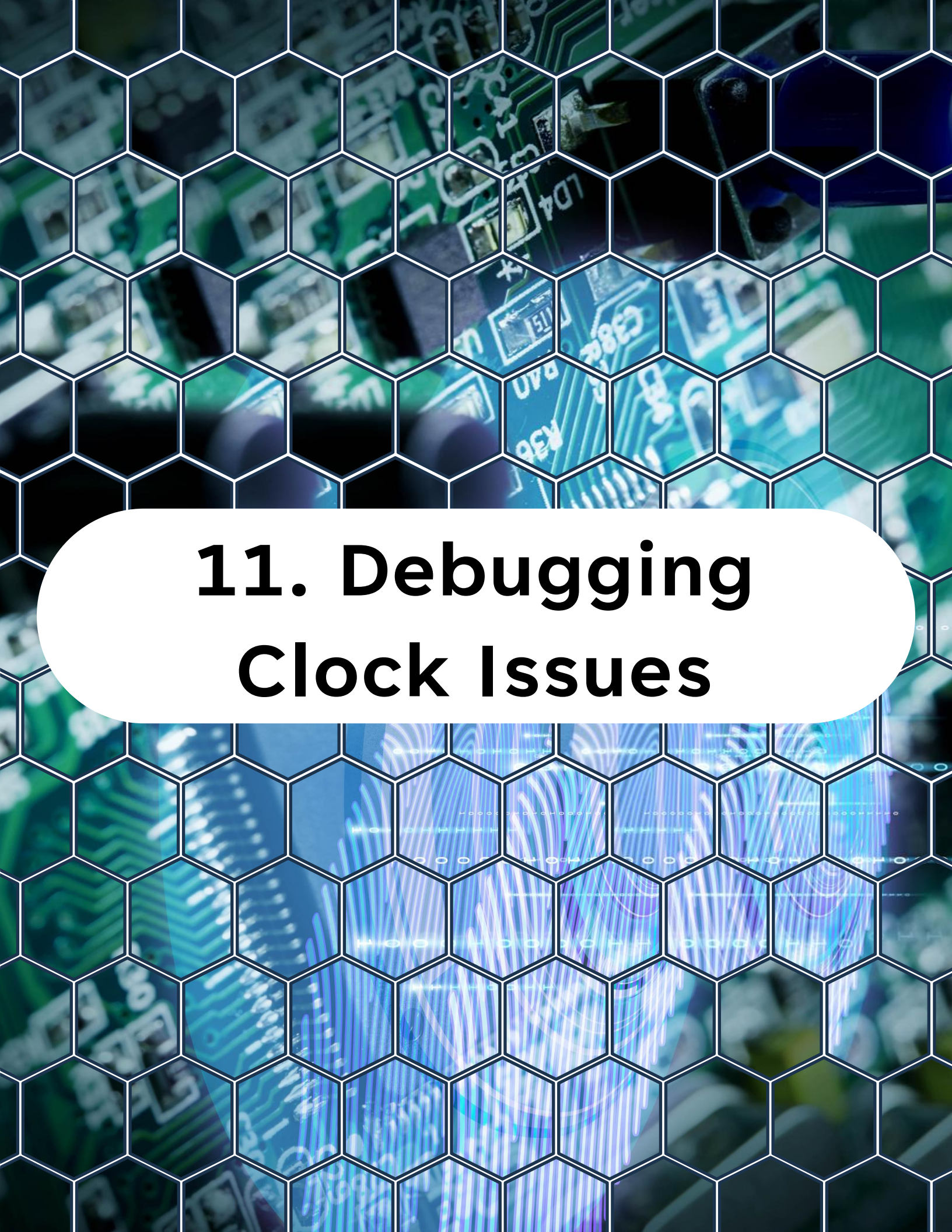
Observe:

- Output frequency
- Stability
- Duty cycle

Then:

- Compare measured frequency with expected CPU clock
- Adjust prescaler or clock source
- Measure again

This feedback loop is how embedded engineers validate assumptions.



11. Debugging Clock Issues

11. Debugging Clock Issues

Common clock-related mistakes:

- Forgetting prescaler is enabled
- Assuming default clock frequency
- Changing clock source without waiting for stability
- Using wrong clock domain for peripherals

Symptoms:

- Wrong baud rate
- Incorrect PWM frequency
- Timers running too fast or too slow

When timing is wrong, check clocks first.

Always.



12. What You Learned Today

12. What You Learned Today

By the end of Day 3, you should understand:

- Where the CPU clock comes from
- How CLKCTRL configures clock sources
- How prescalers affect execution speed
- How peripherals depend on clock domains
- How to verify clock configuration using real hardware

This knowledge is foundational. Every peripheral from now on depends on this.



12. What You Learned Today

12. What You Learned Today

By the end of Day 3, you should understand:

- Where the CPU clock comes from
- How CLKCTRL configures clock sources
- How prescalers affect execution speed
- How peripherals depend on clock domains
- How to verify clock configuration using real hardware

This knowledge is foundational. Every peripheral from now on depends on this.



13. What Comes Next

13. What Comes Next

Day 4 - GPIO at Register Level

Next, you will:

- Fully control GPIO pins
- Learn input, output, pull-ups, inversion
- Build clean, deterministic pin control code
- Use GPIO as a debugging and timing tool

GPIO is simple, but mastering it makes everything else easier.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>